

第八讲：延续价值就是你所需要的一切

异质性主体宏观经济学的计算方法

孙杰

2026 年 5 月 20 日

多伦多大学

神经价值函数迭代 (Neural Value Function Iteration, nVFI)

神经价值函数迭代 (neural Value Function Iteration, nVFI)

大致流程：

1. 用参数为 θ 的神经网络 (neural network) 对 $V^{\text{end}}(x^{\text{end}}, \Lambda^{\text{start}})$ 进行参数化
2. (以某种方式) 从 Λ^{end} 抽样
3. 对每个 Λ^{end} 计算**外层贝尔曼残差** (outer Bellman residual, 定义见下)
4. 更新 θ , 从第 2 步起重复, 直至收敛

外层贝尔曼残差

考虑 Krusell-Smith 模型。

设 $\hat{V}_{\theta}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{end}}, Z)$ 为以 θ 为参数的神经网络。

非正式地说，我们希望尽可能地满足

$$\hat{V}_{\theta}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) - \mathbb{E}_{Z'} \left[\hat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}'}, Z') \mid Z \right] = 0,$$

其中 $\hat{V}^{\text{start}}(x^{\text{end}}; \Lambda^{\text{start}'}, Z')$ 由先向前模拟、再向后迭代得到，所用的是 $\hat{V}_{\theta}^{\text{end}}, Z'$ 。

约定与时序

约定

1. 家庭状态在家庭期之间不变：

$$x_{it}^{\text{end}} = x_{i,t+1}^{\text{start}}, \quad \text{并且} \quad \Lambda_{it}^{\text{end}} = \Lambda_{i,t+1}^{\text{start}}.$$

2. 总量冲击发生在家庭期之间

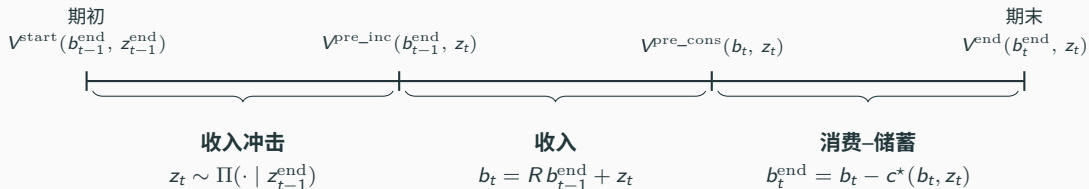
- 即介于 $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 与 $V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}'}, Z')$ 之间

3. 仅依赖于**个体特异性** (idiosyncratic) 状态的值函数**切片** (slice) 或**猜测** (guess) 以 tilde 表示,

$$\tilde{V}^{\text{start}}(x^{\text{start}}), \quad \text{而} \quad V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z).$$

期内时间线：稳态

回顾稳态版本中期内的时序结构：



期内时间线：全局解

加入总量冲击后，我们重新书写（并简化）：

$$\begin{array}{ccc}
 \tilde{V}^{\text{start}}(x^{\text{start}}) & & \tilde{V}^{\text{end}}(x^{\text{end}}) \\
 \equiv V^{\text{start}}(x^{\text{start}}; \Lambda^{\text{start}}, Z) & & \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)
 \end{array}$$

Λ^{start} 家庭期 Λ^{end}

在前面的讲座中，我们学习了如何构造映射

$$\left(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z \right) \mapsto \left(\tilde{V}^{\text{start}}, \Lambda^{\text{end}} \right).$$

记之为 $\Phi \left(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z \right) = \left(\tilde{V}^{\text{start}}, \Lambda^{\text{end}} \right).$

约定：对 Λ^{start} 的依赖

- 模型在家庭期内是确定性的。
- 因此， Λ^{end} 由 Λ^{start} 唯一决定。
- 因此，**在理论上是有效的**：可以写 $V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z)$ 。
- **注意**：此处我尚未就如何在计算机上计算或表示 V^{end} 作出任何陈述！

约定： V^{end} 的偏应用

- 如果我们对 Λ^{start} 与 Z 取条件，则 V^{end} 就只是 x^{start} 的函数。
- 即固定 Λ^{start} 与 Z ，可以定义

$$\tilde{V}^{\text{end}}(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

- 用计算机科学的术语说，这是 V^{end} 在 $(\Lambda^{\text{start}}, Z)$ 处的**偏应用**（partial application）。
- 我们将其写成一个把**函数**映射到**函数**的算子

$$\mathcal{V} : (\Lambda^{\text{start}}, Z) \mapsto \tilde{V}^{\text{end}}$$

$$\mathcal{V}(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv V^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z).$$

我的看法是：**延续价值算子**（continuation value operator） $\mathcal{V}$ 是值函数在带总量冲击的异质性主体模型中的自然推广。

延续价值就是你所需要的一切
(Continuation Value Is All You
Need)

延续价值就是你所需要的一切

假设我们有一个模型，初始总量状态为 $(\Lambda^{\text{start}}, Z)$ 。

再假设我们已经有延续价值算子 $\mathcal{V}$ 。（这正是难的部分。）

那么我们就可以将模型向前模拟：

1. $\tilde{V}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z)$
2. $(\tilde{V}^{\text{start}}, \Lambda^{\text{end}}) = \Phi(\tilde{V}^{\text{end}}, \Lambda^{\text{start}}, Z)$
3. 抽取 $Z' \sim \Pi(\cdot \mid Z)$
4. $(\Lambda^{\text{start}'}, Z') = (\Lambda^{\text{end}}, Z')$

回顾：典型的贝尔曼方程与算子

典型的贝尔曼方程通过下式定义 V :

$$V(b, z) = \max_{c \in [0, b]} u(c) + \beta \mathbb{E}[V(b', z') \mid z]$$

这可以视为贝尔曼算子 $\mathcal{T}$ 的不动点，该算子施行**前瞻**（lookahead）:

$$\mathcal{T}v(b, z) \equiv \max_{c \in [0, b]} u(c) + \beta \mathbb{E}[v(b', z') \mid z]$$

于是贝尔曼方程为

$$V = \mathcal{T}V.$$

下面我定义一个类似的外层贝尔曼方程。

外层贝尔曼方程

真实的 $\mathcal{V}$ 也满足一个递归方程，即**外层贝尔曼方程**：

$$\mathcal{V}^{\text{end}}(x^{\text{end}}; \Lambda^{\text{start}}, Z) = \mathbb{E}_Z \left[\tilde{\mathcal{V}}^{\text{start}'}(x^{\text{end}}) \mid Z \right],$$

其中

$$\tilde{\mathcal{V}}^{\text{start}'} = \Phi_V(\Lambda^{\text{end}}, \tilde{\mathcal{V}}^{\text{end}'}, Z'),$$

$$\tilde{\mathcal{V}}^{\text{end}'} = \mathcal{V}(\Lambda^{\text{end}}, Z'),$$

$$Z' \sim \Pi_Z(\cdot \mid Z),$$

$$\Lambda^{\text{end}} = \Phi_\Lambda(\Lambda^{\text{start}}, \tilde{\mathcal{V}}^{\text{end}}, Z),$$

$$\tilde{\mathcal{V}}^{\text{end}} = \mathcal{V}(\Lambda^{\text{start}}, Z).$$

外层贝尔曼算子

对任意 $\mathcal{V}_\theta$ ，可以定义**外层贝尔曼算子**如下：

$$\mathcal{L}\mathcal{V}_\theta(\Lambda^{\text{start}}, Z)(x^{\text{end}}) \equiv \mathbb{E}_Z \left[\tilde{\mathcal{V}}^{\text{start}'}(x^{\text{end}}) \mid Z \right],$$

其中

$$\tilde{\mathcal{V}}^{\text{start}'} = \Phi_V(\Lambda^{\text{end}}, \tilde{\mathcal{V}}^{\text{end}'}, Z'),$$

$$\tilde{\mathcal{V}}^{\text{end}'} = \mathcal{V}_\theta(\Lambda^{\text{end}}, Z'),$$

$$Z' \sim \Pi_Z(\cdot \mid Z),$$

$$\Lambda^{\text{end}} = \Phi_\Lambda(\Lambda^{\text{start}}, \tilde{\mathcal{V}}^{\text{end}}, Z),$$

$$\tilde{\mathcal{V}}^{\text{end}} = \mathcal{V}_\theta(\Lambda^{\text{start}}, Z).$$

外层贝尔曼残差

真实的**延续价值算子** $\mathcal{V}$ 由下式定义：

$$\mathcal{L}\mathcal{V} = \mathcal{V}.$$

对任意 $\mathcal{V}_\theta$ ，可以定义**外层贝尔曼残差**为

$$\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta,$$

并将逼近 $\mathcal{V}$ 的问题写成

$$\min_{\theta} \|\mathcal{L}\mathcal{V}_\theta - \mathcal{V}_\theta\|^2.$$

实现 CVIAYN

从第七讲继承的内容

第七讲的 K-S 笔记本已经为我们提供了除网络与训练循环之外的一切：

- KSPParams（第七讲第 3 节）。
- ks_prices(K, Z, p)（第七讲第 4 节）。
- 三阶段模拟链 $\text{ks_household_sim} = \text{z_shock} \circ \text{income} \circ \text{savings}$ （第七讲第 8 节）。
- aiyagari_steady_state_at_Z（第七讲第 5 节）——为预训练给出 V_{ss} ，并提供采样器的初始条件 Λ_{ss} 。

我们舍弃的部分：K_evolve + Z_shock 链的尾段（即家庭对总量运动法则 ALM 的信念），以及 V 中的 K -网格。网络取代了这两者。

Φ 在代码中：就是那条模拟链

三阶段链就是我们的 Φ 。给定个体特异性 (b, z) 网格上的任意 $\tilde{V}^{\text{end}}$ ，它

- 向后运行得到 $\tilde{V}^{\text{start}}$ （savings 阶段求解家庭的选择；income / z_shock 阶段组合而成）；
- 向前运行，将 μ^{start} 推到 μ^{end} 。

它单独无法做到的唯一一件事，是提供 $\tilde{V}^{\text{end}}$ 。这正是 ν_θ 所提供的。

网络结构： Λ 如何进入？

我们不能把一个 $N_b \times N_z$ 的场直接塞进一个全连接层而指望学到什么。标准做法 (Han–Yang–E 2025) 是：在 value head 看到 Λ 之前，先把它总结为一个小的可学习向量。

四个子网络：

1. $\text{pop_agg_net} : (b, z) \rightarrow \mathbb{R}^k$ ——对每个 cell 做嵌入。
2. $\text{GM}(\Lambda) = \sum_{(b,z)} \Lambda(b, z) \cdot \text{pop_agg_net}(b, z) \in \mathbb{R}^k$ ——嵌入的人口加权均值。一次矩阵乘法。
3. $V_{\text{pre}} : (b, z) \rightarrow \mathbb{R}^m$ ——value head 用到的 cell 级特征。
4. $V_{\text{post}} : \mathbb{R}^{m+N_z} \rightarrow \mathbb{R}$ ——将 V_{pre} 的特征、(投影后的) GM 与 Z 的 one-hot 相组合。

组合方式是**加法的**： $V_{\text{pre}}(\text{cell}) + \text{gm_post}(\text{GM})$ ，然后再拼接 Z 。

网络在代码中的样子

```
struct VNet{A, P, G, V}
    pop_agg_net::A    # 2 -> gm_dim        per-cell embedding
    V_pre::P          # 2 -> mid            per-cell features
    gm_post::G         # gm_dim -> mid       project GM to mid
    V_post::V          # mid + N_Z -> 1      combine + Z
end

function (net::VNet)( $\Lambda$ , Z_idx, p)
    pop_emb = net.pop_agg_net(HH_FEAT)      # (gm_dim, N_cells)
    gm       = pop_emb * Float32.(vec( $\Lambda$ )) # (gm_dim,)
    h        = net.V_pre(HH_FEAT) .+ net.gm_post(gm) # (mid, N_cells)
    pred     = net.V_post(vcat(h, Z_ROWS[Z_idx])) # (1, N_cells)
    return reshape(pred, p.N_w, length(p.z_grid))
end
```

瓶颈 $k = \text{gm_dim}$ 用一个可学习的 k 维摘要替换了 $N_b N_z$ 维的输入。

在 V_θ 下进行一次内层求解

```
function inner_solve(hh, vnet::VNet,  $\Lambda$ ::AbstractMatrix, Z_idx::Int, p)
    V_end = Float64.(vnet( $\Lambda$ , Z_idx, p)) # N_b x N_z
    env    = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ ),
                                           p.Z_vals[Z_idx], p)...)
    return backward!(hh, V_end, env)
end
```

- 网络在 (Λ, Z) 处预测 $\hat{V}_\theta^{\text{end}}$ 。
- 以价格 $(r, w) = \text{ks_prices}(K(\Lambda), Z, p)$ 构造 env。价格仅通过 $K = \int b d\Lambda$ 依赖于 Λ ——这是生产技术的设定，而非网络架构的特性。
- 在三阶段链上做一次向后扫掠 $\Rightarrow \tilde{V}^{\text{start}}$ 。

前瞻算子 $\mathcal{L}V_\theta$

```
function lookahead(hh, vnet,  $\Lambda$ _start, Z_idx, p)
    # Forward one period under V_theta to get Lambda_end.
    V_end_now = Float64.(vnet( $\Lambda$ _start, Z_idx, p))
    env_now    = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ _start),
                                                    p.Z_vals[Z_idx], p)...)

    backward!(hh, V_end_now, env_now)
     $\Lambda$ _end = forward!(hh,  $\Lambda$ _start)

    # Backward at each next-period Z; take expectation over Z.
    V_end_pred = zeros(p.N_w, length(p.z_grid))
    for Z_next in 1:N_Z
        V_start_next = inner_solve(hh, vnet,  $\Lambda$ _end, Z_next, p)
        V_end_pred .+= p.P_Z[Z_idx, Z_next] .* V_start_next
    end
    return V_end_pred
end
```

刚才发生了什么

我们在一个样本 Λ_i^{start} (总量索引为 Z_i) 处计算了 $\mathcal{L}V_\theta$:

1. 网络在 $(\Lambda_i^{\text{start}}, Z_i)$ 处预测 $\hat{V}_\theta^{\text{end}}$ 。
2. Φ 把 Λ_i^{start} 向前推一步 $\rightarrow \Lambda_i^{\text{end}}$ 。
3. 对每个 Z_{next} , 网络在 $(\Lambda_i^{\text{end}}, Z_{\text{next}})$ 处预测 $\hat{V}_\theta^{\text{end}}$ 。经过 Φ 的向后扫掠得到 $V_{\text{next}}^{\text{start}}$ 。
4. 用 $P_Z[Z_i, \cdot]$ 对 Z_{next} 取期望。

结果是一个 (b, z) 形状的数组 $\widehat{V}_{i, \cdot}^{\text{end}}$ 。这就是我们的训练标签。

训练损失

```
function bellman_residual_loss(vnet, batch, p, hh)
    L = 0.0
    for ( $\Lambda$ _start, Z_idx) in batch
        target = lookahead(hh, vnet,  $\Lambda$ _start, Z_idx, p)
        pred    = vnet( $\Lambda$ _start, Z_idx, p)
        L += mean((pred .- Float32.(target)).^2) / var_scale(target)
    end
    return L / length(batch)
end
```

对标签使用 stop-gradient。 前瞻只算一次，并作为固定的目标处理。我们不对 Φ （期内求解器）做反向传播。只有 V_θ 直接对 θ 的依赖才进入梯度。

对 Λ^{start} 的采样

```
function sample_ergodic(hh, vnet, p; T=200, burn=100, every=2, rng=...)
     $\Lambda$ , Z_idx = copy( $\Lambda_{ss}$ ), 1
    batch = Tuple{Matrix{Float64}, Int}[]
    for t in 1:T
        V_end = Float64.(vnet( $\Lambda$ , Z_idx, p))
        env = make_env(hh; ks_prices(integrate_K(hh,  $\Lambda$ ),
                                                p.Z_vals[Z_idx], p)...)
        backward!(hh, V_end, env);  $\Lambda$  = forward!(hh,  $\Lambda$ )
        Z_idx = sample_Z(Z_idx, p.P_Z, rng)
        t > burn && (t - burn) % every == 0 && push!(batch, (copy( $\Lambda$ ), Z_idx))
    end
    return batch
end
```

MCMC 风格：在当前的 V_θ 下进行模拟，烧入 (burn in)，然后子采样。

训练循环

```
function train!(vnet, p, hh; epochs=50, lr=1e-3, batch_kwargs=(; T=200, burn=100))  
    opt = Flux.setup(Adam(lr), vnet)  
    hist = Float64[]  
    for epoch in 1:epochs  
        batch = sample_ergodic(p, hh, vnet; batch_kwargs...)  
        loss, grads = Flux.withgradient(vnet) do net  
            bellman_residual_loss(net, batch, p, hh)  
        end  
        Flux.update!(opt, vnet, grads[1])  
        push!(hist, loss)  
    end  
    return hist  
end
```

初始化很关键

随机初始化的 V_θ 通常会让期内求解失败（如出现数值上无界的消费等）。

预训练。让网络在一个 Λ 的云（对 Λ_{ss} 加扰动）以及所有 Z 上去拟合确定性的值函数 V_{ss} 。这不是因为 V_{ss} 在任何 $\Lambda \neq \Lambda_{ss}$ 处是正确的——它并不正确——而是因为它是一个合理的常数基线，能在残差训练循环开始挪动一切之前，把 V_θ 调到合适的量级与形状。

成本低，不依赖第七讲。

运行

```
p    = KSParams()
hh    = ks_household_2d(p)
vnet = VNet()
pretrain_to_ss!(vnet, V_ss, p)

hist = train!(vnet, hh, p; epochs=50)
plot(hist; yaxis=:log, lw=2,
      xlabel="epoch", ylabel="outer Bellman residual")
```

诚实声明。 CPU 上 50 个 epoch 只是演示，而非已收敛的解。真实的运行需要在 GPU 上跑数千乃至上万个 epoch。详见第 7 节。

解读结果

训练曲线

- 外层贝尔曼残差（对数尺度）随训练 epoch 的变化。
- 应当看到一条带噪声但总体下降的曲线。
- 在真实运行中（Peking 2025 deck 第 5 节）：神经网络达到约 $10^{-4.4}$ 的相对方差；K-S 基线则在约 $10^{-3.2}$ 处停滞。

差距才是要点：收敛后，在同一问题上， V_θ 对残差方差的解释优于 K-S，而且无需任何参数化先验形式。

“外层贝尔曼残差”作为误差意味着什么

残差小意味着 V_θ 近似是 $\mathcal{L}$ 的不动点。

由压缩映射论证（标准结果，当 $\mathcal{L}$ 是 β -压缩时成立），这意味着 $\|V_\theta - V\| \leq \frac{1}{1-\beta} \|V_\theta - \mathcal{L}V_\theta\|$ 。残差小 $\Rightarrow$ 离真值近。

警示。对本设定而言， $\mathcal{L}$ 的压缩性质属于研究层面的问题；论文附录给出了一些条件。对我们的 K-S 基线而言，经验上的压缩性是成立的。

与 K-S 的比较

将 V_θ 与第七讲的 K-S 基线进行比较，有三种方式：

1. **值函数 MSE**：在共同的 (b, z, Λ_t, Z_t) 模拟路径上比较。K-S 在 K_t 处取值；CVIAYN 在 Λ_t 处取值。
2. **策略 MSE**：从二者分别抽取 b' ，再做比较。
3. **Den Haan 误差**：分别在两种方法下模拟总量 K_t 路径。在此基线校准下，CVIAYN 与 K-S 在该指标上仅相差噪声，这是因为 K-S 的先验形式 (ansatz) 恰好在该基线校准下接近真实情况（见第七讲第 6 节）。

有意思的差距出现在 (1) 与 (2)，并集中在尾部状态（罕见的 Λ 、罕见的 Z ）上：K-S 外推的是线性 ALM；CVIAYN 外推的是网络。

K-S 等价性在何处崩溃

K-S 在基线校准下看上去与 CVIAYN 难以区分。这是因为 K 的条件运动法则恰好近似对数线性——而且我们为定价所关心的人口边际分布 K 恰好是一个标量。

CVIAYN 拉开差距的场景：

- 多资产（流动性差的财富）：动态不再是对数线性。
- 离散选择（行业、职业）：占有份额不是 K 的矩。
- 空间模型：跨地区的分布无法由总量来概括。

在以上每一种情形中，K-S 都需要一个手工构造的、追踪额外矩的先验形式；CVIAYN 只需扩展 GM 的输入。

推广

这在哪里重要

标量 K 先验形式无法奏效的模型：

- 搜寻-匹配 (vacancy distribution)。
- OLG (横截面的年龄结构)。
- 空间模型 (人口在何处，而非仅仅有多少)。
- 多资产 (流动性 vs. 流动性差的构成)。
- 离散选择 (职业构成)。

带 GM 输入的 CVIAYN 可以推广到所有这些情形而不改动算法本身。配方沿着良好理解的轴线扩展：更多的 cell 特征、按地区的 GM、按年龄的 GM。

GM 架构如何扩展

按地区的 GM（空间/城市模型）。对每个地区 ℓ 用其自身的 Λ 切片构造一个 GM_ℓ 。再用一个“聚合的聚合”（aggregate-of-aggregates）网络在地区之间做汇总。

年龄分层（OLG）。对每个队列（cohort）运行一个独立的 `pop_agg_net`，或者共享权重，把年龄作为额外的 cell 特征喂入。

多资产。状态空间增加一个维度；`pop_agg_net` 的输入变为 (b_{liq}, b_{illiq}, z) 。其余不变。

非人口状态。另启一个 `env_net`，把外生总量状态嵌入一个并行的 GM；按同样的方式加到 cell 级特征上。

优化后的参考实现

笔记本中的训练循环虽诚实，但属于教学版本。一个生产级别的实现位于本仓库中：

- `reference_materials/
example_usage/stochastic_transition/code/
stochastic_transition/`

它在今天的结构之上还增加了：按地区的 GM + 聚合的聚合 head；用于非人口状态的并行 `env_net`；余弦退火学习率；低显存的梯度累积；模拟路径上对期内求解的缓存；按地区的反向传播配以共享的 GM。

算法本身没变，新增的全是工程化的管线（plumbing）。

单步前瞻只是一个旋钮

第 5 节中的前瞻是单期的：在 Λ^{start} 处求解 Φ ，抽取 Z_{next} ，在 $\Lambda^{\text{start}'}$ 处再次求解 Φ ，然后取期望。

n 步前瞻。在算子内部做 n 轮 (Φ, Ω) 。等价于施加 $\mathcal{L}^n$ 。

- 优点：残差目标的偏差更小（前瞻越深 $\rightarrow$ 标签越真）。
- 缺点：成本随 n 线性增长；方差随 Z 路径数增长。

笔记本中使用 $n = 1$ 。改成更高的 n 只需一行修改。

三段简短的附录，分别讨论：

- 时序约定为何重要，
- 为何对前瞻标签使用 `stop-gradient`，
- 相关文献。

时序约定为何重要

将值函数拆分为 V^{start} 与 V^{end} 是让 $\mathcal{V}$ 成为一个干净对象的关键。

若不做拆分： $V(x^{\text{start}}, \Lambda^{\text{start}})$ 依赖于 Λ^{start} ，同时也隐含地对 t 期内发生的一切进行预报。这使得“网络学到了什么”与“ Φ 计算了什么”难以分离。

做拆分之后： V^{end} 位于下一次总量冲击之前那一刻。期内的一切由 Φ 处理；跨期的一切由 $\mathcal{V}$ 处理。

为何对标签使用 stop-gradient

标签 $\widehat{V}^{\text{end}}_i$ 本身就是用 V_θ （通过前瞻）算出来的。也可以对其反向传播。

我们不这样做的理由：

- 每次前瞻包含数次 Φ 的运行，而 Φ 中涉及网格搜索、单调搜索技巧以及其他不可微的环节。
- “损失即不动点残差”这一表述是 ML 处理价值迭代的标准做法；SARSA、Q-learning 等都对 bootstrap 目标做停止梯度（stop gradient）。
- 经验上，半梯度（semi-gradient）能稳定训练。

投影 / 局部摄动法： Bhandari et al. (2023), Bilal (2023), Auclert et al. (2021), Winberry (2018)。

深度策略逼近： Han et al. (2024), Azinovic et al. (2022), Maliar et al. (2021), Fernández-Villaverde et al. (2023), Gu et al. (2024)。

非神经的值函数逼近： Krusell & Smith (1998), Hull (2015)。

广义矩架构 (generalized moments)： Han, Yang & E (2025)。

CVIAYN 用深度学习扩展了第三类方法，并借用了第四类方法中的 GM 构造。